

LAPORAN PRAKTIKUM
STRUKTUR DATA
IMPLEMENTASI STRUKTUR DATA QUEUE PADA JAVA

Disusun Oleh :

Rahmat Ananda Nazar

2511532008

Dosen Pengampu : Dr. Wahyudi, S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANADALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa berkat rahmat dan karunianya penulis dapat menyelesaikan laporan praktikum Struktur Data pekan 4 dengan judul “Impelementasi Struktur Data Queue Pada Java” ini dengan baik. Pada laporan praktikum pekan satu ini akan membahas mengenai penerapa konsep ADT (Abstract Data Type) spesifiknya membahas Struktur data Queue.

Penulis menyadari bahwa laporan ini tidak terlepas dari kesalahan dan kekurangan baik dari penulisan dan pembahasan. Oleh karena itu penulis ingin mengucapkan terimakasih kepada dosen pengampu Bapak Dr. Wahyudi, S.T M.T. dan asisten praktikum Muhammad Galid Averro yang telah memberikan arahan selama proses praktikum dan proses penyusunan laporan praktikum. Penulis berharap laporan ini dapat bermanfaat bagi pembaca dalam memahami struktur data queue dan penerapannya dalam java.

DAFTAR PUSTAKA

KATA PENGANTAR.....	i
DAFTAR PUSTAKA	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 QueueArray dan Driver.....	2
2.1.1 Kode Program dan Output	2
2.1.2 Penjelasan Program	3
2.2 Reverse Data	4
2.2.1 Kode Program dan Output	4
2.2.2 Penjelasan Program	4
2.3 Queue LinkedList.....	5
2.3.1 Kode Program dan Output.....	5
2.3.2 Penjelasan Program.....	5
2.4 Iterasi Queue	6
2.4.1 Kode Program dan Output	6
2.4.2 Penjelasan Program	6
BAB III PENUTUP	7
3.1 Kesimpulan	7
DAFTAR PUSTAKA	8

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, penyusunan data secara efisien merupakan aspek krusial untuk membangun sistem yang terstruktur dan mudah dikelola. Salah satu konsep fundamental dalam mata kuliah Struktur Data adalah Abstract Data Type (ADT), Praktikum ini berfokus pada penerapan konsep lanjutan untuk ADT melalui pemahaman struktur data Queue. Melalui implementasi ini, diharapkan praktikan dapat memahami cara menggunakan struktur data Queue pada program

1.2 Tujuan Praktikum

Implementasi Struktur data Queue agar mampu menerapkannya pada program untuk membuat sistem yang relevan dan sesuai dengan kebutuhan pengguna yang ada.

1.3 Manfaat Praktikum

Praktikum ini memberikan manfaat pemahaman teoretis melalui penerapan langsung struktur data Queue pada program. Melalui kegiatan ini, praktikan dapat mengasah keterampilan teknis dalam membangun struktur data yang rapi menggunakan kelas ADT, mengelola alur program yang dinamis dengan Queue, praktikum ini melatih kemampuan pemecahan masalah (problem solving) dan logika berpikir sistematis saat mengintegrasikan berbagai fungsi ke dalam satu aplikasi yang fungsional dan terstruktur.

BAB II

PEMBAHASAN

2.1 QueueArray dan Driver

2.1.1 Kode Program dan Output

```
1 package pekan4_2511532008;
2
3 public class QueueArray_2511532008 {
4     int front_2008, rear_2008, size_2008;
5     int capacity_2008;
6     int array_2008[];
7     public QueueArray_2511532008(int capacity) {
8         this.capacity_2008 = capacity;
9         front_2008 = this.size_2008 = 0;
10        rear_2008 = capacity - 1;
11        array_2008 = new int[this.capacity_2008];
12    }
13    boolean isFull_2008 (QueueArray_2511532008 queue) {
14        return (queue.size_2008 == queue.capacity_2008);
15    }
16    boolean isEmpty_2008 (QueueArray_2511532008 queue_2008) {
17        return (queue_2008.size_2008 == 0);
18    }
19    void enqueue_2008(int item_2008) {
20        if (isFull_2008(this))
21            return ;
22        this.rear_2008 = (this.rear_2008 + 1) % this.capacity_2008;
23        this.array_2008[this.rear_2008] = item_2008;
24        this.size_2008 = this.size_2008 + 1;
25        System.out.println(item_2008 + "enqueued to queue");
26    }
27    int dequeue_2008() {
28        if (isEmpty_2008(this))
29            return Integer.MIN_VALUE;
30        int item = this.array_2008[this.front_2008 ];
31        this.front_2008 = (this.front_2008 + 1) % this.capacity_2008;
32        this.size_2008 = size_2008 ;
33        return item;
34    }
35    int front_2008() {
36        if (isEmpty_2008(this))
37            return Integer.MIN_VALUE;
38        return this.array_2008[this.front_2008];
39    }
40    int rear_2008() {
41        if (isEmpty_2008(this))
42            return Integer.MIN_VALUE;
43        return this.array_2008[this.rear_2008];
44    }
45    void display_2008() {
46        int i;
47        if (front_2008 == size_2008) {
48            System.out.println("Antrian Kosong");
49            return;
50        }
51        for (i = front_2008; i < rear_2008; i++) {
52            System.out.print("%d <-- " + array_2008[i]);
53        }
54        return;
55    }
56 }
```

Gambar 2.1

```

1 package pekan4_2511532008;
2
3 public class QueueArrayDriver_2511532008 {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7         QueueArray_2511532008 queue_2008 = new QueueArray_2511532008(1000);
8
9         queue_2008.enqueue_2008(10);
10        queue_2008.enqueue_2008(20);
11        queue_2008.enqueue_2008(30);
12        queue_2008.enqueue_2008(40);
13
14        System.out.println("Item di depan = " + queue_2008.front_2008());
15        System.out.println("Item paling belakang = " + queue_2008.rear_2008());
16        System.out.println("Tampilan queue ");
17        queue_2008.display_2008();
18        System.out.println("");
19        System.out.println(queue_2008.dequeue_2008() + "hapus dari queue ");
20        System.out.println("item di depan " + queue_2008.front_2008());
21        System.out.println("item di belakang " + queue_2008.rear_2008());
22        System.out.println("tampilan setelah di hapus; ");
23        queue_2008.display_2008();
24    }
25
26 }

```

Gambar 2.2

```

10enqueued to queue
20enqueued to queue
30enqueued to queue
40enqueued to queue
Item di depan = 10
Item paling belakang = 40
Tampilan queue
%d <-- 10%d <-- 20%d <-- 30
10hapus dari queue
item di depan 20
item di belakang 40
tampilan setelah di hapus;
%d <-- 20%d <-- 30

```

Gambar 2.3

2.1.2 Penjelasan Program

Pada program ini membuat konstruktor dengan atribut dan beberapa method, seperti method void untuk menambahkan antrian dan juga method untuk menampilkan isi dari antrian. Terdapat juga method non void atau fungsi int dan boolean untuk melakukan penghapusan antrian

2.2 Reverse Data

2.2.1 Kode Program dan Output

```
1 package pekan4_2511532008;
2 import java.util.*;
3 public class ReversseData_2511532008 {
4     public static void main(String[] args) {
5         // TODO Auto-generated method stub
6         Queue<Integer> q_2008 = new LinkedList<Integer>();
7         q_2008.add(1);
8         q_2008.add(2);
9         q_2008.add(3);
10        System.out.println("Sebelum reverse " + q_2008);
11        Stack<Integer> s_2008 = new Stack<Integer>();
12        while (!q_2008.isEmpty()) {
13            s_2008.push(q_2008.remove());
14        }
15        while (!s_2008.isEmpty()) {
16            q_2008.add(s_2008.pop());
17        }
18        System.out.println("Setelah reverse : " + q_2008);
19    }
```

Gambar 2.4

```
Sebelum reverse [1, 2, 3]
Setelah reverse : [3, 2, 1]
```

Gambar 2.5

2.2.2 Penjelasan Program

Pada program ini adalah program untuk membalikkan urutan elemen dalam sebuah antrean (Queue) dengan memanfaatkan Stack. Proses diawali dengan memasukkan data ke dalam Queue, kemudian seluruh elemen tersebut dipindahkan ke dalam Stack satu per satu

2.3 Queue LinkedList

2.3.1 Kode Program dan Output

```
1 package pekan4_2511532008;
2 import java.util.LinkedList;
3 import java.util.Queue;
4 public class QueueLinkedList_2511532008 {
5     public static void main(String[] args) {
6         Queue<Integer> q_2008 = new LinkedList<>();
7         for (int i = 0; i < 6; i++) {
8             q_2008.add(i);
9         }
10        System.out.println ("Elemen Antrian " + q_2008);
11        int hapus_2008 = q_2008.remove ();
12        System.out.println ("Hapus elemen = " + hapus_2008);
13        System.out.println (q_2008);
14        int depan_2008 = q_2008.peek();
15        System.out.println ("kepala antrian = " + depan_2008);
16        int banyak_2008 = q_2008.size();
17        System.out.println ("Size Antrian = " + banyak_2008);
18    }
```

Gambar 2.6

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
kepala antrian = 1
Size Antrian = 5
```

Gambar 2.7

2.3.2 Penjelasan Program

Pada program ini tahapan diawali dengan pengisian elemen angka melalui iterasi, diikuti oleh proses penghapusan elemen terdepan menggunakan metode `remove()`. Selanjutnya, program melakukan inspeksi untuk melihat elemen yang berada di posisi kepala antrian tanpa menghapusnya lewat metode `peek()`, serta diakhiri dengan penghitungan total elemen yang tersisa dalam antrian melalui metode `size()`.

2.4 Iterasi Queue

2.4.1 Kode Program dan Output

```
1 package pekan4_2511532008;
2 import java.util.*;
3 public class IterasiQueue_2511532008 {
4     public static void main(String [] args ) {
5         // TODO Auto-generated method stub
6         Queue<String> q_2008 = new LinkedList<>();
7         q_2008.add("Praktikum");
8         q_2008.add("Struktur");
9         q_2008.add("Data");
10        q_2008.add("Dan");
11        q_2008.add("Algoritma");
12        Iterator<String> iterator = q_2008.iterator();
13        while (iterator.hasNext()) {
14            System.out.print(iterator.next() + " |");
15        }
16    }
17 }
18
```

Gambar 2.8

```
Praktikum Struktur Data Dan Algoritma
```

Gambar 2.9

2.4.2 Penjelasan Program

Pada program ini kita membuat objek antrian diinisialisasi dan diisi dengan sejumlah data string, proses melalui struktur perulangan, metode hasNext() berfungsi untuk memvalidasi keberadaan data selanjutnya dalam urutan, sedangkan metode next() digunakan untuk mengakses sekaligus menampilkan nilai setiap elemen secara berurutan hingga seluruh isi antrian berhasil diproses.

BAB III

PENUTUP

3.1 Kesimpulan

Struktur data Queue merupakan instrumen fundamental dalam pemrograman yang mengedepankan prinsip FIFO, di mana elemen yang pertama kali masuk akan menjadi yang pertama kali diproses. Melalui operasi dasar seperti penambahan dan penghapusan. Hal ini menjadikannya struktur queue sangat penting dan sangat efisien untuk menangani proses-proses yang memerlukan urutan kronologis yang akurat.

DAFTAR PUSTAKA

W3Schools, “Java Tutorial,” 2026. [Daring]. Tersedia pada:
<https://www.w3schools.com/java/default.asp>.

TUGAS PRAKTIKUM

Kode Program :

```
1 package pekan4_2511532008;
2
3 import java.util.*;
4
5 public class AntrianLoket_2511532008 {
6     String[] queue_2008;
7     int front_2008;
8     int rear_2008;
9     int max_2008;
10    public AntrianLoket_2511532008(int kapasitasMaksimal_2008) {
11        max_2008 = kapasitasMaksimal_2008;
12        queue_2008 = new String[max_2008];
13        front_2008 = -1;
14        rear_2008 = -1;
15    }
16    public boolean isEmpty_2008() {
17        return front_2008 == -1;
18    }
19    public boolean isFull_2008() {
20        return rear_2008 == max_2008 - 1;
21    }
22    public void enqueue_2008(String nama_2008) {
23        if (isFull_2008()) {
24            System.out.println("Antrian penuh!");
25        } else {
26            if (isEmpty_2008()) {
27                front_2008 = 0;
28            }
29            rear_2008++;
30            queue_2008[rear_2008] = nama_2008;
31            System.out.println("Data berhasil ditambahkan ke antrian");
32        }
33    }
34    public void dequeue_2008() {
35        if (isEmpty_2008()) {
36            System.out.println("Antrian kosong!");
37        } else {
38            System.out.println(queue_2008[front_2008] + " telah dilayani");
39            if (front_2008 == rear_2008) {
40                front_2008 = -1;
41                rear_2008 = -1;
42            } else {
43                front_2008++;
44            }
45        }
46    }
47    public void display_2008() {
48        if (isEmpty_2008()) {
49            System.out.println("Antrian kosong!");
50        } else {
51            System.out.println("Isi antrian:");
52            int nomor_2008 = 1;
53            for (int i_2008 = front_2008; i_2008 <= rear_2008; i_2008++) {
54                System.out.println(nomor_2008 + ". " + queue_2008[i_2008]);
55                nomor_2008++;
56            }
57        }
58    }
59 }
```

```

58     }
59     public void Reverse_2008() {
60         if (isEmpty_2008()) {
61             System.out.println("Antrian kosong!");
62             return;
63         }
64         int i_2008 = front_2008;
65         int j_2008 = rear_2008;
66         while (i_2008 < j_2008) {
67             String temp_2008 = queue_2008[i_2008];
68             queue_2008[i_2008] = queue_2008[j_2008];
69             queue_2008[j_2008] = temp_2008;
70             i_2008++;
71             j_2008--;
72         }
73     }
74     public static void main(String[] args) {
75         Scanner input_2008 = new Scanner(System.in);
76         AntrianLoket_2511532008 loket_2008 = new AntrianLoket_2511532008(10);
77         int pilihan_2008;
78         do {
79             System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
80             System.out.println("1. Tambah Antrian");
81             System.out.println("2. Hapus Antrian");
82             System.out.println("3. Tampilkan Antrian");
83             System.out.println("4. Reverse");
84             System.out.println("5. Keluar");
85             System.out.print("Pilih menu: ");
86             pilihan_2008 = input_2008.nextInt();
87             input_2008.nextLine();
88             switch (pilihan_2008) {
89                 case 1:
90                     System.out.print("Masukkan nama pelanggan: ");
91                     String nama_2008 = input_2008.nextLine();
92                     loket_2008.enqueue_2008(nama_2008);
93                     break;
94                 case 2:
95                     loket_2008.dequeue_2008();
96                     break;
97                 case 3:
98                     loket_2008.display_2008();
99                     break;
100                 case 4:
101                     loket_2008.Reverse_2008();
102                     loket_2008.display_2008();
103                     break;
104                 case 5:
105                     break;
106                 default:
107                     System.out.println("Pilihan tidak valid! Silakan masukkan angka 1-5.");
108             }
109         } while (pilihan_2008 != 5);
110     }
111 }
112

```

Contoh Output :

```
Console X
AntrianLoket_2511532008 [Java Application] C:\Users\rahma\.p2\pool\plugins\org.eclipse

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Joni
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Asep
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Josep
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Joni
2. Asep
3. Josep
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Isi antrian:
1. Josep
2. Asep
3. Joni

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Josep telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Asep
2. Joni

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu:
```